DETAILS

NAME: THOTA.DEEPTHI

SECTION: CSE 1

ROLL NO: 26000062

1. University Scholarship Allocation

Problem Statement

Design a Python program to evaluate scholarship applicants using aggregate marks, subject-wise minimum marks, attendance, and income criteria. The program should identify eligible students, rank them based on their performance, and resolve ties using predefined rules.

Objectives

1. To calculate the aggregate marks of each student.
2. To check the minimum marks required in each subject.
3. To verify attendance and family income eligibility.
4. To identify eligible scholarship applicants.
5. To rank eligible students based on their aggregate marks.
6. To resolve ties using attendance as the tie-breaking criterion.

Assumptions

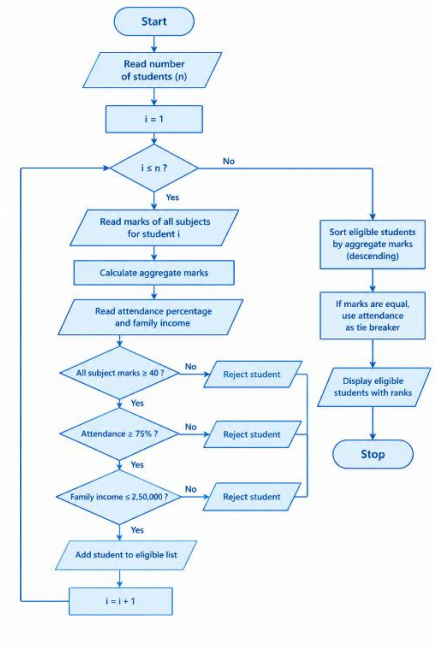
- There are 3 subjects.
- Minimum marks required in each subject = 40.
- Minimum attendance required = 75%.
- Maximum eligible annual family income = ₹2,50,000.
- Students with higher aggregate marks get higher ranks.
- If two students have the same aggregate, the student with higher attendance gets the higher rank.

Algorithm

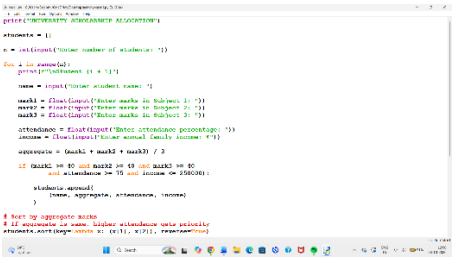
1. Start.
2. Read the number of students.
3. For each student, read the name, marks in three subjects, attendance, and annual family income.
4. Calculate the aggregate marks.
5. Check whether the student has scored at least 40 marks in every subject.
6. Check whether attendance is at least 75%.

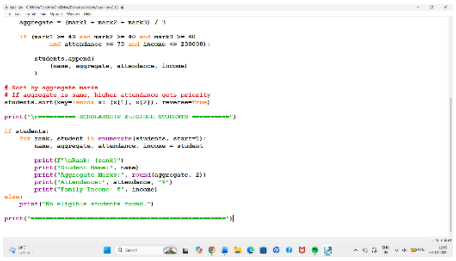
7. Check whether annual family income is not more than ₹2,50,000.
8. If all conditions are satisfied, add the student to the eligible list.
9. Sort eligible students by aggregate marks in descending order.
10. If aggregate marks are equal, sort them by attendance in descending order.
11. Display the ranked list of eligible students.
12. Stop.

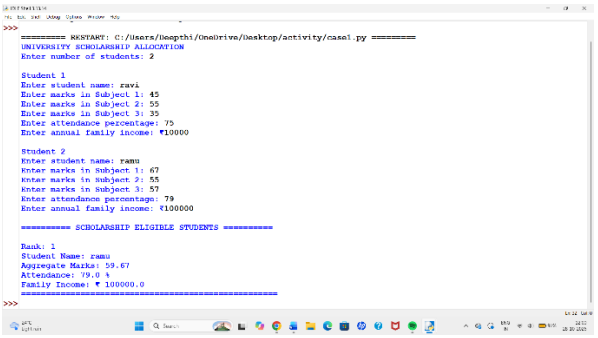
Flowchart



Pseudocode







START

Read number of students

FOR each student
  Read name
  Read marks in three subjects
  Read attendance
  Read annual family income

  Calculate aggregate marks

  IF subject marks are at least 40
    AND attendance is at least 75%
    AND income is at most 250000
  THEN
     Add student to eligible list
  END IF
END FOR

Sort eligible students by aggregate marks
in descending order.

If aggregate marks are equal,
sort by attendance in descending order.

Display ranked eligible students

STOP

Python Source Code

```
students = []

n = int(input("Enter number of students: "))

for i in range(n):
    print("\nStudent", i + 1)

    name = input("Enter name: ")
    m1 = int(input("Enter Subject 1 marks: "))
    m2 = int(input("Enter Subject 2 marks: "))
    m3 = int(input("Enter Subject 3 marks: "))
    attendance = float(input("Enter attendance percentage: "))
    income = float(input("Enter annual family income: "))

    aggregate = m1 + m2 + m3

    if (m1 >= 40 and m2 >= 40 and m3 >= 40
        and attendance >= 75
        and income <= 250000):

      students.append(
        [name, aggregate, attendance, income]
      )
```

```
students.sort(key=lambda x: (x[1], x[2]), reverse=True)


print("\n--- SCHOLARSHIP ELIGIBLE STUDENTS ---")


if len(students) == 0:
    print("No eligible students found.")
else:
    rank = 1


    for student in students:
        print("Rank:", rank)
        print("Name:", student[0])
        print("Aggregate:", student[1])
        print("Attendance:", student[2], "%")
        print("Income: Rs.", student[3])
        print()
        rank += 1
```

Test Cases

Test Case	Subject Marks	Attendance	Income	Expected Result
1	80, 75, 90	85%	₹2,00,000	Eligible
2	35, 80, 90	85%	₹2,00,000	Not Eligible
3	80, 75, 90	70%	₹2,00,000	Not Eligible
4	80, 75, 90	85%	₹3,00,000	Not Eligible
5	Same aggregate for two students	Different attendance	Eligible	Higher attendance gets higher rank

Sample Output

Enter number of students: 3

--- SCHOLARSHIP ELIGIBLE STUDENTS ---

Rank: 1

Name: Ravi

Aggregate: 260

Attendance: 90.0 %

Income: Rs. 180000.0

Rank: 2

Name: Sita

Aggregate: 245

Attendance: 82.0 %

Income: Rs. 220000.0

Conclusion

The Python program successfully evaluates scholarship applicants based on **aggregate marks, subject-wise minimum marks, attendance, and income criteria**. It identifies eligible students, ranks them according to their academic perform

2. Progressive Electricity Billing

Problem Statement

Design a Python program to calculate household electricity bills using multiple consumption slabs, fixed charges, surcharges, and rebates. The program should compare electricity bills across several months and identify unusual increases in consumption or billing.

Objectives

1. To calculate electricity charges based on different consumption slabs.
2. To add fixed charges to the electricity bill.
3. To apply surcharges when applicable.
4. To apply rebates when applicable.
5. To compare electricity bills for different months.
6. To identify unusual increases in monthly bills.

Assumptions

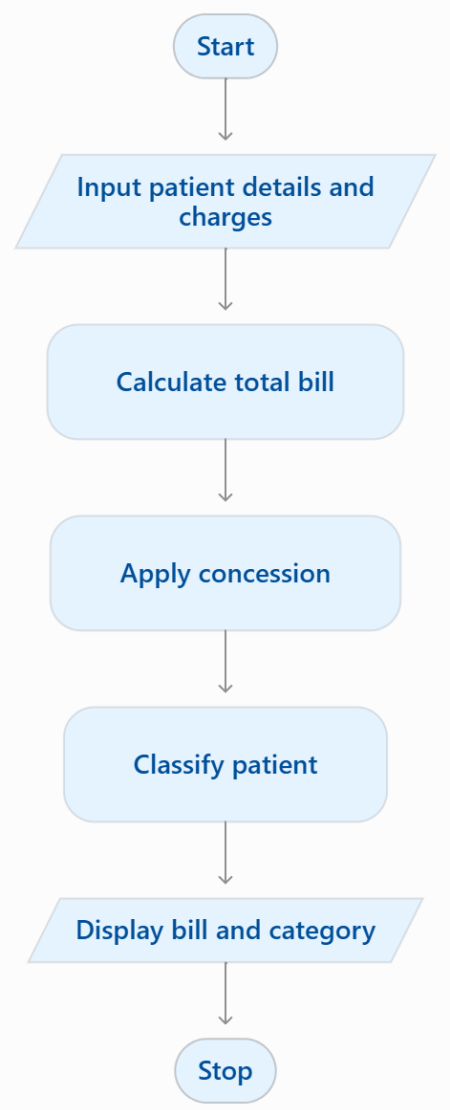
- Consumption is measured in units.
- Slab rates are:
    - First 100 units → ₹2 per unit
    - Next 100 units → ₹3 per unit
    - Above 200 units → ₹5 per unit
- Fixed charge = ₹50 per month.
- If consumption is above 300 units, a 10% surcharge is applied.
- If consumption is below 100 units, a 5% rebate is applied.
- An increase of more than 20% compared with the previous month is considered unusual.

Algorithm

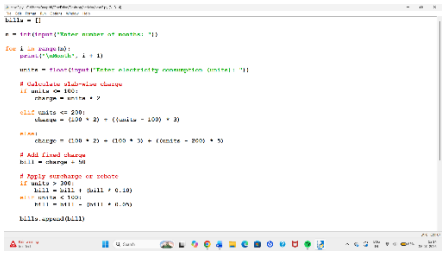
1. Start.
2. Read the number of months.
3. For each month, read the electricity consumption in units.
4. Calculate the charge using the progressive slabs.

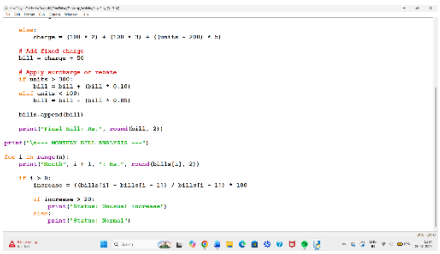
5. Add the fixed charge.
6. Apply a 10% surcharge if consumption is above 300 units.
7. Apply a 5% rebate if consumption is below 100 units.
8. Calculate the final bill.
9. Compare the current month's bill with the previous month's bill.
10. If the increase is more than 20%, mark it as an unusual increase.
11. Display the monthly consumption and bill.
12. Stop.

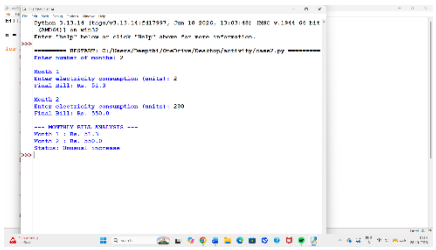
Flowchart



Pseudocode







START

Read number of months

FOR each month

Read electricity consumption

IF units <= 100

charge = units * 2

ELSE IF units <= 200

charge = 100 * 2 + (units - 100) * 3

ELSE

charge = 100 * 2 + 100 * 3 + (units - 200) * 5

END IF

Add fixed charge of 50

IF units > 300

Apply 10% surcharge

```
  ELSE IF units < 100
    Apply 5% rebate
  END IF

  Calculate final bill

  IF previous bill exists
    Calculate percentage increase

    IF increase > 20%
      Mark as unusual increase
    END IF
  END IF

  Display monthly bill

END FOR

STOP
```

Python Source Code

Python Source Code

```
bills = []

n = int(input("Enter number of months: "))

for i in range(n):
  print("\nMonth", i + 1)
```

```
    units = float(input("Enter electricity consumption (units): "))

    # Calculate slab-wise charge
    if units <= 100:
        charge = units * 2

    elif units <= 200:
        charge = (100 * 2) + ((units - 100) * 3)

    else:
        charge = (100 * 2) + (100 * 3) + ((units - 200) * 5)

    # Add fixed charge
    bill = charge + 50

    # Apply surcharge or rebate
    if units > 300:
        bill = bill + (bill * 0.10)
    elif units < 100:
        bill = bill - (bill * 0.05)

    bills.append(bill)

    print("Final Bill: Rs.", round(bill, 2))

print("\n--- MONTHLY BILL ANALYSIS ---")

for i in range(n):
```

```
    print("Month", i + 1, ": Rs.", round(bills[i], 2))

    if i > 0:
        increase = ((bills[i] - bills[i - 1]) / bills[i - 1]) * 100

        if increase > 20:
            print("Status: Unusual increase")
        else:
            print("Status: Normal")
```

Test Cases

Test Case	Units	Calculation	Expected Result
1	80	80 × 2 + ₹50 − 5% rebate	Valid bill
2	150	100 × 2 + 50 × 3 + ₹50	Valid bill
3	250	100 × 2 + 100 × 3 + 50 × 5 + ₹50	Valid bill
4	350	Slab charge + ₹50 + 10% surcharge	Valid bill
5	Previous ₹300, Current ₹400	Increase = 33.33%	Unusual increase

Sample Output

Enter number of months: 3

Month 1

Enter electricity consumption (units): 80

Final Bill: Rs. 202.5

Month 2

Enter electricity consumption (units): 150

Final Bill: Rs. 300.0

--- MONTHLY BILL ANALYSIS ---

Month 1 : Rs. 202.5

Month 2 : Rs. 300.0

Status: Unusual increase

Conclusion

The program successfully calculates progressive electricity bills using **consumption slabs, fixed charges, surcharges, and rebates**. It also compares monthly bills and identifies unusual increases in billing.

3. Hospital Billing and Patient Classification

Problem Statement

Design a Python program to calculate a patient's hospital bill using consultation, diagnostic, and treatment charges. Apply concessions based on specified conditions, validate billing constraints, and classify patients according to a given payment policy.

Objectives

1. To calculate the total consultation charges.
2. To calculate diagnostic and treatment charges.
3. To apply concessions based on eligibility conditions.
4. To validate the patient's billing details.
5. To calculate the final payable amount.
6. To classify patients according to their payment status.

Assumptions

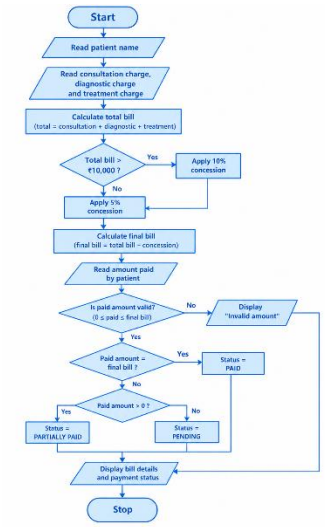
- Consultation charge = ₹500.
- Diagnostic charge depends on the selected diagnostic service.
- Treatment charge depends on the treatment category.
- A 10% concession is given if the total bill is above ₹10,000.
- A 5% concession is given to patients with valid concession eligibility when the bill is ₹10,000 or below.
- The final bill cannot be negative.
- Payment classification:
    - **Paid** – full amount has been paid.
    - **Partially Paid** – some amount has been paid.
    - **Pending** – no amount has been paid.

Algorithm

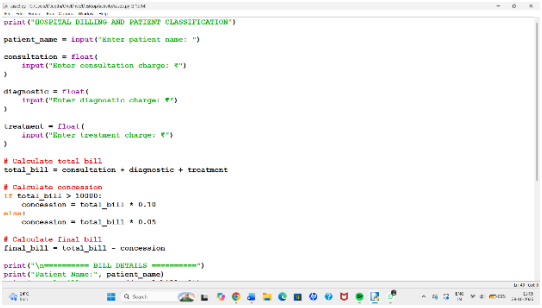
1. Start.
2. Read patient name.
3. Read consultation charge.
4. Read diagnostic charge.

5. Read treatment charge.
6. Calculate the total bill.
7. Check whether the bill is greater than ₹10,000.
8. Apply the appropriate concession.
9. Calculate the final payable amount.
10. Read the amount paid.
11. Validate that the amount paid is not negative and does not exceed the final bill.
12. Classify the patient's payment status.
13. Display the complete bill.
14. Stop.

Flowchart



Pseudocode



```
print("HOSPITAL BILLING AND PATIENT CLASSIFICATION")

patient_name = input("Enter patient name: ")

consultation = float(
    input("Enter consultation charge: ₹")
)

diagnostic = float(
    input("Enter diagnostic charge: ₹")
)

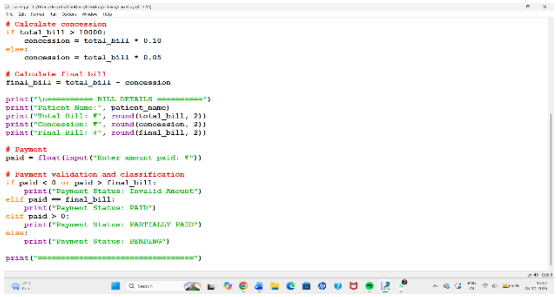
treatment = float(
    input("Enter treatment charge: ₹")
)

# Calculate total bill
total_bill = consultation + diagnostic + treatment

# Calculate concession
if total_bill > 10000:
    concession = total_bill * 0.10
else:
    concession = total_bill * 0.05

# Calculate final bill
final_bill = total_bill - concession

print("\n========== BILL DETAILS ==========")
print("Patient Name:", patient_name)
```



```
# Calculate concession
if total_bill > 10000:
    concession = total_bill * 0.10
else:
    concession = total_bill * 0.05

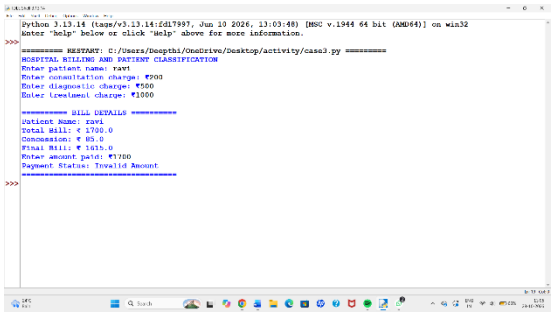
# Calculate final bill
final_bill = total_bill - concession

print("\n========== BILL DETAILS ==========")
print("Patient Name:", patient_name)
print("Total Bill: ₹", round(total_bill, 2))
print("Concession: ₹", round(concession, 2))
print("Final Bill: ₹", round(final_bill, 2))

# Payment
paid = float(input("Enter amount paid: ₹"))

# Payment validation and classification
if paid < 0 or paid > final_bill:
    print("Payment Status: Invalid Amount")
elif paid == final_bill:
    print("Payment Status: PAID")
elif paid > 0:
    print("Payment Status: PARTIALLY PAID")
else:
    print("Payment Status: PENDING")

print("==================================")
```



START

Read patient name

Read consultation charge

Read diagnostic charge

Read treatment charge

total = consultation + diagnostic + treatment

IF total > 10000

  concession = total * 10 / 100

ELSE

  concession = total * 5 / 100

END IF

final_bill = total - concession

Read amount paid

IF amount paid < 0 OR amount paid > final_bill

  Display "Invalid payment"

ELSE

```
  IF amount paid == final_bill
    status = "Paid"
  ELSE IF amount paid > 0
    status = "Partially Paid"
  ELSE
    status = "Pending"
  END IF
END IF

Display total, concession, final bill and status

STOP
```

Python Source Code

```
print("HOSPITAL BILLING SYSTEM")

name = input("Enter patient name: ")

consultation = float(input("Enter consultation charge: "))
diagnostic = float(input("Enter diagnostic charge: "))
treatment = float(input("Enter treatment charge: "))

# Calculate total bill
total = consultation + diagnostic + treatment

# Apply concession
if total > 10000:
  concession = total * 0.10
else:
```

```
    concession = total * 0.05

final_bill = total - concession

# Read payment
paid = float(input("Enter amount paid: "))

print("\n--- BILL DETAILS ---")
print("Patient Name:", name)
print("Total Bill: Rs.", round(total, 2))
print("Concession: Rs.", round(concession, 2))
print("Final Bill: Rs.", round(final_bill, 2))

# Validate payment
if paid < 0 or paid > final_bill:
    print("Payment Status: Invalid Payment")
else:
    if paid == final_bill:
        status = "Paid"
    elif paid > 0:
        status = "Partially Paid"
    else:
        status = "Pending"

    balance = final_bill - paid

    print("Amount Paid: Rs.", round(paid, 2))
    print("Balance: Rs.", round(balance, 2))
```

```
print("Payment Status:", status)
```

Test Cases

Test Case	Total Charges	Amount Paid	Expected Result
1	₹8,000	₹7,600	Partially Paid
2	₹8,000	₹7,600	5% concession applied
3	₹12,000	₹10,800	10% concession applied
4	₹10,000	₹9,500	Partially Paid
5	₹12,000	₹13,000	Invalid Payment

Sample Output

HOSPITAL BILLING SYSTEM

Enter patient name: Ravi

Enter consultation charge: 500

Enter diagnostic charge: 3500

Enter treatment charge: 7000

Enter amount paid: 9900

--- BILL DETAILS ---

Patient Name: Ravi

Total Bill: Rs. 11000.0

Concession: Rs. 1100.0

Final Bill: Rs. 9900.0

Amount Paid: Rs. 9900.0

Balance: Rs. 0.0

Payment Status: Paid

Conclusion

The Python program successfully calculates hospital bills using **consultation, diagnostic, and treatment charges**. It applies conditional concessions, validates payments, calculates the remaining balance, and classifies the pati

4. Employee Payroll and Deduction Analysis

Problem Statement

Design a Python program to calculate an employee's gross salary using basic pay, allowances, overtime, and incentives. Apply progressive deductions and generate a formatted salary statement showing the employee's salary details and net salary.

Objectives

1. To calculate the employee's gross salary.
2. To calculate allowances based on basic pay.
3. To calculate overtime earnings.
4. To include incentives in the salary.
5. To apply progressive deductions.
6. To calculate the employee's net salary.
7. To display a formatted salary statement.

Assumptions

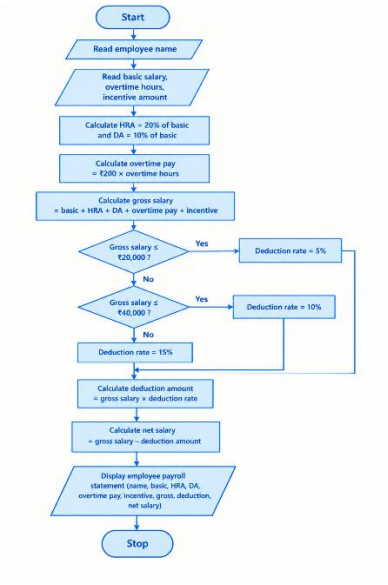
- Basic pay is entered by the user.
- House Rent Allowance (HRA) = 20% of basic pay.
- Dearness Allowance (DA) = 10% of basic pay.
- Overtime is calculated at ₹200 per hour.
- Incentive amount is entered by the user.
- Progressive deduction:
    - Gross salary up to ₹20,000 → 5% deduction
    - ₹20,001 to ₹40,000 → 10% deduction
    - Above ₹40,000 → 15% deduction
- Net salary = Gross salary − Deduction.

Algorithm

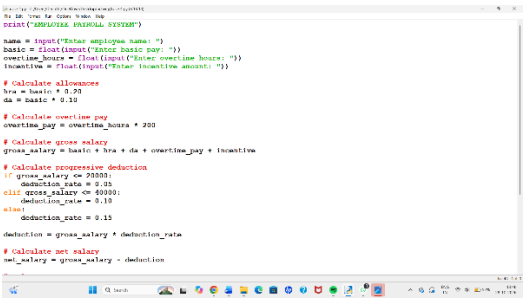
1. Start.
2. Read employee name.
3. Read basic pay.
4. Calculate HRA as 20% of basic pay.

5. Calculate DA as 10% of basic pay.
6. Read overtime hours.
7. Calculate overtime pay at ₹200 per hour.
8. Read incentive amount.
9. Calculate gross salary.
10. Apply the appropriate progressive deduction rate.
11. Calculate net salary.
12. Display the salary statement.
13. Stop.

Flowchart



Pseudocode



```
print("EMPLOYEE PAYROLL SYSTEM")

name = input("Enter employee name: ")
basic = float(input("Enter basic pay: "))
overtime_hours = float(input("Enter overtime hours: "))
incentive = float(input("Enter incentive amount: "))

# Calculate allowances
hra = basic * 0.20
da = basic * 0.10

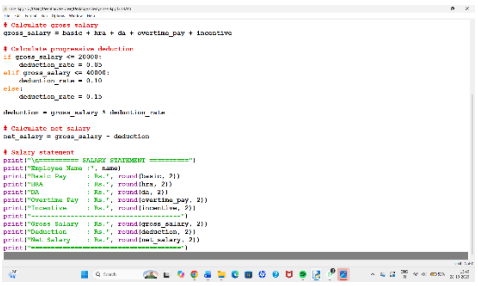
# Calculate overtime pay
overtime_pay = overtime_hours * 200

# Calculate gross salary
gross_salary = basic + hra + da + overtime_pay + incentive

# Calculate progressive deduction
if gross_salary <= 20000:
    deduction_rate = 0.05
elif gross_salary <= 40000:
    deduction_rate = 0.10
else:
    deduction_rate = 0.15

deduction = gross_salary * deduction_rate

# Calculate net salary
net_salary = gross_salary - deduction
```



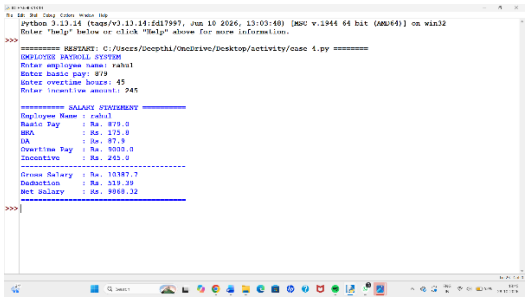
```
# Calculate gross salary
gross_salary = basic + hra + da + overtime_pay + incentive

# Calculate progressive deduction
if gross_salary <= 20000:
    deduction_rate = 0.05
elif gross_salary <= 40000:
    deduction_rate = 0.10
else:
    deduction_rate = 0.15

deduction = gross_salary * deduction_rate

# Calculate net salary
net_salary = gross_salary - deduction

# Salary statement
print("\n========== SALARY STATEMENT ==========")
print("Employee Name :", name)
print("Basic Pay     : Rs.", round(basic, 2))
print("HRA           : Rs.", round(hra, 2))
print("DA            : Rs.", round(da, 2))
print("Overtime Pay  : Rs.", round(overtime_pay, 2))
print("Incentive     : Rs.", round(incentive, 2))
print("--------------------------------------")
print("Gross Salary  : Rs.", round(gross_salary, 2))
print("Deduction     : Rs.", round(deduction, 2))
print("Net Salary    : Rs.", round(net_salary, 2))
print("======================================")
```



START

Read employee name

Read basic pay

Read overtime hours

Read incentive

HRA = basic pay * 20 / 100

DA = basic pay * 10 / 100

Overtime pay = overtime hours * 200

Gross salary = basic pay + HRA + DA

+ overtime pay + incentive

IF gross salary <= 20000

deduction = gross salary * 5 / 100

ELSE IF gross salary <= 40000

deduction = gross salary * 10 / 100

ELSE

deduction = gross salary * 15 / 100

END IF

Net salary = gross salary - deduction

Display salary statement

STOP

Python Source Code

```
print("EMPLOYEE PAYROLL SYSTEM")

name = input("Enter employee name: ")
basic = float(input("Enter basic pay: "))
overtime_hours = float(input("Enter overtime hours: "))
incentive = float(input("Enter incentive amount: "))

# Calculate allowances
hra = basic * 0.20
da = basic * 0.10

# Calculate overtime pay
overtime_pay = overtime_hours * 200

# Calculate gross salary
gross_salary = basic + hra + da + overtime_pay + incentive

# Calculate progressive deduction
if gross_salary <= 20000:
   deduction_rate = 0.05
elif gross_salary <= 40000:
```

```
    deduction_rate = 0.10
else:
    deduction_rate = 0.15

deduction = gross_salary * deduction_rate

# Calculate net salary
net_salary = gross_salary - deduction

# Salary statement
print("\n========== SALARY STATEMENT ==========")
print("Employee Name :", name)
print("Basic Pay    : Rs.", round(basic, 2))
print("HRA          : Rs.", round(hra, 2))
print("DA           : Rs.", round(da, 2))
print("Overtime Pay  : Rs.", round(overtime_pay, 2))
print("Incentive    : Rs.", round(incentive, 2))
print("--------------------------------------")
print("Gross Salary  : Rs.", round(gross_salary, 2))
print("Deduction    : Rs.", round(deduction, 2))
print("Net Salary   : Rs.", round(net_salary, 2))
print("======================================")
```

Test Cases

Test Case	Basic Pay	Overtime	Incentive	Expected Result
1	₹15,000	5 hrs	₹500	5% deduction
2	₹25,000	5 hrs	₹1,000	10% deduction
3	₹35,000	10 hrs	₹2,000	15% deduction

4	₹10,000	0 hrs	₹0	5% deduction
5	₹40,000	10 hrs	₹5,000	15% deduction

Sample Output

EMPLOYEE PAYROLL SYSTEM

Enter employee name: Ravi

Enter basic pay: 25000

Enter overtime hours: 5

Enter incentive amount: 1000

========== SALARY STATEMENT ==========

Employee Name : Ravi

Basic Pay : Rs. 25000.0

HRA : Rs. 5000.0

DA : Rs. 2500.0

Overtime Pay : Rs. 1000.0

Incentive : Rs. 1000.0

Gross Salary : Rs. 34500.0

Deduction : Rs. 3450.0

Net Salary : Rs. 31050.0

Conclusion

The program successfully calculates an employee's **gross salary, allowances, overtime pay, incentives, progressive deduction, and net salary**. It also generates a clear and formatted salary statement.

CASE STUDY 5: MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION

1. Problem Statement

Develop a Python program to evaluate several mobile plans based on the user's usage, validity requirements, and budget constraints. The program should calculate excess usage charges, check plan eligibility, and identify the most economical eligible plan.

2. Objectives

- To compare multiple mobile plans.
- To check whether a plan satisfies the user's usage requirements.
- To calculate excess usage charges.
- To verify whether the total cost is within the user's budget.
- To identify the most economical eligible plan.

3. Assumptions

Since the PDF does not specify numerical plan rates, the following assumptions are used to make the program runnable:

- Each plan has a fixed monthly cost.
- Each plan provides a fixed amount of data.
- Extra data is charged per GB.
- Each plan has a validity period.
- The user provides required data, validity, and maximum budget.
- A plan is eligible only if:
    - Data provided ≥ required data
    - Validity ≥ required validity
    - Total cost ≤ budget

Sample Plans

Plan	Cost	Data	Validity	Extra Data Charge
Plan A	₹299	20 GB	28 days	₹20/GB
Plan B	₹399	30 GB	56 days	₹15/GB

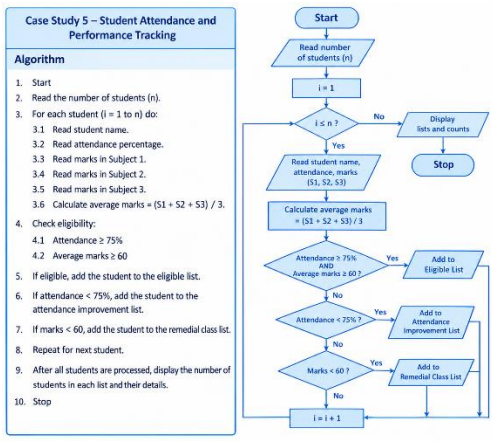
Plan C ₹499 50 GB 84 days ₹10/GB

4. Algorithm

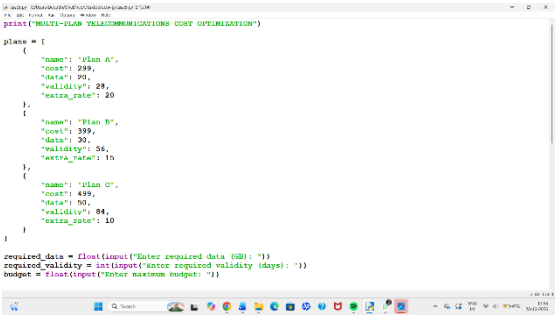
1. Start.
2. Store details of available mobile plans.
3. Read the user's required data.
4. Read the required validity.
5. Read the maximum budget.
6. For each plan:
    - Check whether the plan provides sufficient data.
    - Check whether the validity

1. Read the required validity.
2. Read the maximum budget.
3. For each plan:
    - Check whether the plan provides sufficient data.
    - Check whether the validity is sufficient.
    - Calculate excess usage charges if required data is greater than included data.
    - Calculate total cost.
    - Check whether total cost is within the budget.
4. Add all eligible plans to a list.
5. Find the plan with the minimum total cost.
6. Display all eligible plans.
7. Display the most economical plan.
8. Stop.

5. Flowchart



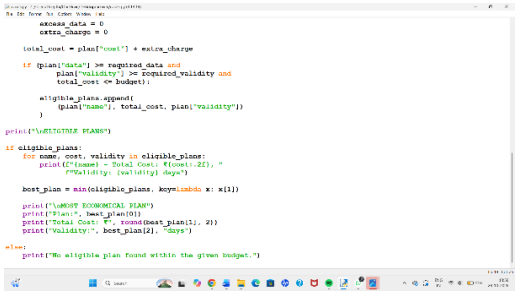
7. Pseudocode



```
print("MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION")

plans = [
    {
        "name": "Plan A",
        "cost": 299,
        "data": 90,
        "validity": 28,
        "extra_rate": 20
    },
    {
        "name": "Plan B",
        "cost": 399,
        "data": 30,
        "validity": 56,
        "extra_rate": 15
    },
    {
        "name": "Plan C",
        "cost": 499,
        "data": 50,
        "validity": 84,
        "extra_rate": 10
    }
]

required_data = float(input("Enter required data (GB): "))
required_validity = int(input("Enter required validity (days): "))
budget = float(input("Enter maximum budget: "))
```



```
        excess_data = 0
        extra_charge = 0

    total_cost = plan["cost"] + extra_charge

    if (plan["data"] >= required_data and
            plan["validity"] >= required_validity and
            total_cost <= budget):

        eligible_plans.append(
            (plan["name"], total_cost, plan["validity"])
        )

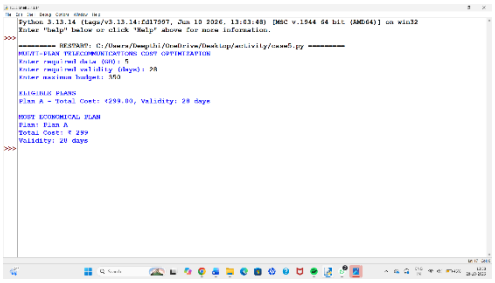
print("\nELIGIBLE PLANS")

if eligible_plans:
    for name, cost, validity in eligible_plans:
        print(f"{name} - Total Cost: ₹{cost:.2f}, "
              f"Validity: {validity} days")

    best_plan = min(eligible_plans, key=lambda x: x[1])

    print("\nMOST ECONOMICAL PLAN")
    print("Plan:", best_plan[0])
    print("Total Cost: ₹", round(best_plan[1], 2))
    print("Validity:", best_plan[2], "days")

else:
    print("No eligible plan found within the given budget.")
```



```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/Deepthi/OneDrive/Desktop/activity/case5.py ==========
MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION
Enter required data (GB): 5
Enter required validity (days): 28
Enter maximum budget: 350

ELIGIBLE PLANS
Plan A - Total Cost: ₹299.00, Validity: 28 days

MOST ECONOMICAL PLAN
Plan: Plan A
Total Cost: ₹ 299
Validity: 28 days
>>>
```

START

Store mobile plan details

INPUT required_data

INPUT required_validity

INPUT budget

FOR each plan:

IF required_data > plan_data:

excess_data = required_data - plan_data

extra_charge = excess_data × extra_rate

ELSE:

extra_charge = 0

total_cost = plan_cost + extra_charge

IF plan_data >= required_data AND

validity >= required_validity AND

total_cost <= budget:

Add plan to eligible list

IF eligible list is not empty:

Find plan with minimum total cost

Display eligible plans

Display most economical plan

ELSE:

Display "No eligible plan found"

STOP

7. Python Source Code

```
print("MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION")

plans = [
```

```
  {
    "name": "Plan A",
    "cost": 299,
    "data": 20,
    "validity": 28,
    "extra_rate": 20
  },
  {
    "name": "Plan B",
    "cost": 399,
    "data": 30,
    "validity": 56,
    "extra_rate": 15
  },
  {
    "name": "Plan C",
    "cost": 499,
    "data": 50,
    "validity": 84,
    "extra_rate": 10
  }
]

required_data = float(input("Enter required data (GB): "))
required_validity = int(input("Enter required validity (days): "))
budget = float(input("Enter maximum budget: "))

eligible_plans = []
```

```
for plan in plans:

    if required_data > plan["data"]:
        excess_data = required_data - plan["data"]
        extra_charge = excess_data * plan["extra_rate"]
    else:
        excess_data = 0
        extra_charge = 0

    total_cost = plan["cost"] + extra_charge

    if (plan["data"] >= required_data and
            plan["validity"] >= required_validity and
            total_cost <= budget):

        eligible_plans.append(
            (plan["name"], total_cost, plan["validity"])
        )

print("\nELIGIBLE PLANS")

if eligible_plans:
    for name, cost, validity in eligible_plans:
        print(f"{name} - Total Cost: ₹{cost:.2f}, "
              f"Validity: {validity} days")

    best_plan = min(eligible_plans, key=lambda x: x[1])
```

```
    print("\nMOST ECONOMICAL PLAN")
    print("Plan:", best_plan[0])
    print("Total Cost: ₹", round(best_plan[1], 2))
    print("Validity:", best_plan[2], "days")

else:
    print("No eligible plan found within the given budget.")
```

8. Test Cases

Test Case	Required Data	Validity	Budget	Expected Result
TC1	15 GB	28 days	₹350	Plan A
TC2	25 GB	56 days	₹500	Plan B
TC3	40 GB	84 days	₹600	Plan C
TC4	60 GB	84 days	₹500	No eligible plan

9. Sample Output

```
MULTI-PLAN TELECOMMUNICATIONS COST OPTIMIZATION

Enter required data (GB): 25
Enter required validity (days): 56
Enter maximum budget: 500

ELIGIBLE PLANS
Plan B - Total Cost: ₹399.00, Validity: 56 days

MOST ECONOMICAL PLAN
Plan: Plan B
Total Cost: ₹399.0
```

Validity: 56 days

10. Conclusion

The Python program successfully compares multiple telecommunications plans based on data requirements, validity, and budget. It calculates additional charges and identifies the most economical plan among the eligible

CASE STUDY 6: RAILWAY FARE AND RESERVATION CALCULATOR

1. Problem Statement

Develop a Python program to calculate railway fares for multiple passengers based on travel distance, travel class, age-based concessions, reservation charges, and cancellation penalties.

2. Objectives

- To calculate railway fare based on distance.
- To apply different fares for different travel classes.
- To provide age-based concessions.
- To add reservation charges.
- To calculate cancellation penalties.
- To generate the final fare for each passenger.

3. Assumptions

The PDF does not specify numerical fare rates, concession percentages, or cancellation penalties. The following values are therefore used as assumptions for a runnable program.

Distance Slabs

Distance	Base Fare
Up to 100 km	₹1.50/km
101–300 km	₹1.25/km
Above 300 km	₹1.00/km

Class Multipliers

Class	Multiplier
Sleeper	1.0
AC 3-Tier	1.5
AC 2-Tier	2.0

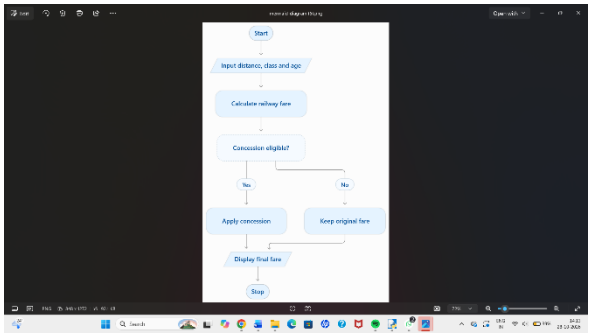
Other Rules

- Children below 5 years: Free.
- Age 5–12 years: 50% concession.
- Senior citizens aged 60 or above: 40% concession.
- Reservation charge: ₹50 per passenger.
- Cancellation penalty: 20% of the fare.
- Cancellation is calculated before reservation charges.

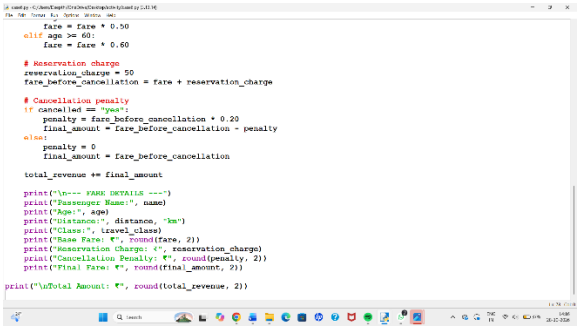
4. Algorithm

1. Start.
2. Read the number of passengers.
3. For each passenger, read:
    - Name
    - Age
    - Distance
    - Travel class
    - Cancellation status
4. Calculate the base fare according to the distance slab.
5. Apply the class multiplier.
6. Apply age-based concession.
7. Add reservation charge.
8. If the ticket is cancelled, calculate cancellation penalty.
9. Calculate the final amount.
10. Display the fare details for each passenger.
11. Stop.

5. Flowchart



7. Pseudocode



```
        fare = fare * 0.50
    elif age >= 60:
        fare = fare * 0.60

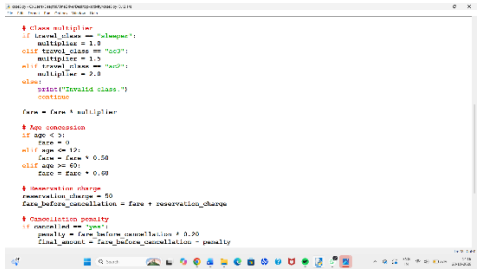
    # Reservation charge
    reservation_charge = 50
    fare_before_cancellation = fare + reservation_charge

    # Cancellation penalty
    if cancelled == "yes":
        penalty = fare_before_cancellation * 0.20
        final_amount = fare_before_cancellation - penalty
    else:
        penalty = 0
        final_amount = fare_before_cancellation

    total_revenue += final_amount

    print("\n--- FARE DETAILS ---")
    print("Passenger Name:", name)
    print("Age:", age)
    print("Distance:", distance, "km")
    print("Class:", travel_class)
    print("Base Fare: ₹", round(fare, 2))
    print("Reservation Charge: ₹", reservation_charge)
    print("Cancellation Penalty: ₹", round(penalty, 2))
    print("Final Fare: ₹", round(final_amount, 2))

print("\nTotal Amount: ₹", round(total_revenue, 2))
```



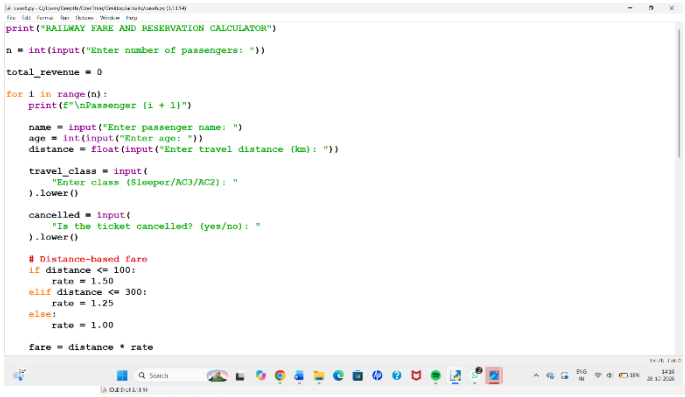
```
# Class multiplier
if travel_class == "sleeper":
    multiplier = 1.0
elif travel_class == "ac3":
    multiplier = 1.5
elif travel_class == "ac2":
    multiplier = 2.0
else:
    print("Invalid class.")
    continue

fare = fare * multiplier

# Age concession
if age < 5:
    fare = 0
elif age <= 12:
    fare = fare * 0.50
elif age >= 60:
    fare = fare * 0.60

# Reservation charge
reservation_charge = 50
fare_before_cancellation = fare + reservation_charge

# Cancellation penalty
if cancelled == "yes":
    penalty = fare_before_cancellation * 0.20
    final_amount = fare_before_cancellation - penalty
```



```
print("RAILWAY FARE AND RESERVATION CALCULATOR")

n = int(input("Enter number of passengers: "))

total_revenue = 0

for i in range(n):
    print(f"\nPassenger {i + 1}")

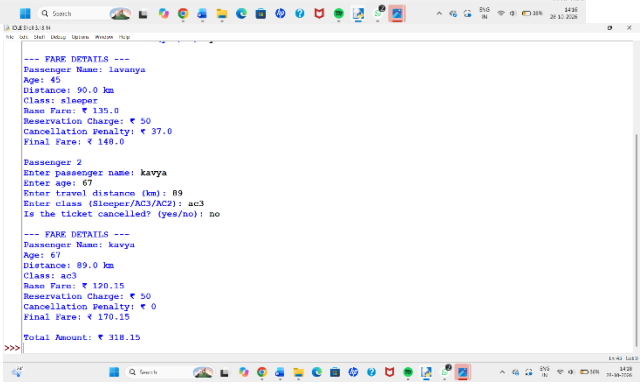
    name = input("Enter passenger name: ")
    age = int(input("Enter age: "))
    distance = float(input("Enter travel distance (km): "))

    travel_class = input(
        "Enter class (Sleeper/AC3/AC2): "
    ).lower()

    cancelled = input(
        "Is the ticket cancelled? (yes/no): "
    ).lower()

    # Distance-based fare
    if distance <= 100:
        rate = 1.50
    elif distance <= 300:
        rate = 1.25
    else:
        rate = 1.00

    fare = distance * rate
```



```
--- FARE DETAILS ---
Passenger Name: lavanya
Age: 45
Distance: 90.0 km
Class: sleeper
Base Fare: ₹ 135.0
Reservation Charge: ₹ 50
Cancellation Penalty: ₹ 37.0
Final Fare: ₹ 148.0

Passenger 2
Enter passenger name: kavya
Enter age: 67
Enter travel distance (km): 89
Enter class (Sleeper/AC3/AC2): ac3
Is the ticket cancelled? (yes/no): no

--- FARE DETAILS ---
Passenger Name: kavya
Age: 67
Distance: 89.0 km
Class: ac3
Base Fare: ₹ 120.15
Reservation Charge: ₹ 50
Cancellation Penalty: ₹ 0
Final Fare: ₹ 170.15

Total Amount: ₹ 318.15
>>>
```

START

INPUT number_of_passengers

```
FOR each passenger:

  INPUT name
  INPUT age
  INPUT distance
  INPUT class
  INPUT cancellation_status

  IF distance <= 100:
    rate = 1.50
  ELSE IF distance <= 300:
    rate = 1.25
  ELSE:
    rate = 1.00

  base_fare = distance × rate

  Apply class multiplier

  IF age < 5:
    fare = 0
  ELSE IF age <= 12:
    fare = fare × 0.50
  ELSE IF age >= 60:
    fare = fare × 0.60

  Add reservation charge
```

```
  IF cancelled:
    penalty = fare × 0.20
    final_amount = fare - penalty
  ELSE:
    final_amount = fare

  DISPLAY final amount

STOP
```

7. Python Source Code

```
print("RAILWAY FARE AND RESERVATION CALCULATOR")

n = int(input("Enter number of passengers: "))

total_revenue = 0

for i in range(n):
  print(f"\nPassenger {i + 1}")

  name = input("Enter passenger name: ")
  age = int(input("Enter age: "))
  distance = float(input("Enter travel distance (km): "))

  travel_class = input(
    "Enter class (Sleeper/AC3/AC2): "
  ).lower()

  cancelled = input(
```

```
    "Is the ticket cancelled? (yes/no): "
).lower()

# Distance-based fare
if distance <= 100:
    rate = 1.50
elif distance <= 300:
    rate = 1.25
else:
    rate = 1.00

fare = distance * rate

# Class multiplier
if travel_class == "sleeper":
    multiplier = 1.0
elif travel_class == "ac3":
    multiplier = 1.5
elif travel_class == "ac2":
    multiplier = 2.0
else:
    print("Invalid class.")
    continue

fare = fare * multiplier

# Age concession
if age < 5:
```

```
    fare = 0
elif age <= 12:
    fare = fare * 0.50
elif age >= 60:
    fare = fare * 0.60

# Reservation charge
reservation_charge = 50
fare_before_cancellation = fare + reservation_charge

# Cancellation penalty
if cancelled == "yes":
    penalty = fare_before_cancellation * 0.20
    final_amount = fare_before_cancellation - penalty
else:
    penalty = 0
    final_amount = fare_before_cancellation

total_revenue += final_amount

print("\n--- FARE DETAILS ---")
print("Passenger Name:", name)
print("Age:", age)
print("Distance:", distance, "km")
print("Class:", travel_class)
print("Base Fare: ₹", round(fare, 2))
print("Reservation Charge: ₹", reservation_charge)
print("Cancellation Penalty: ₹", round(penalty, 2))
```

```
    print("Final Fare: ₹", round(final_amount, 2))


print("\nTotal Amount: ₹", round(total_revenue, 2))
```

8. Test Cases

Test Case	Age	Distance	Class	Cancelled	Expected Result
TC1	25	100 km	Sleeper	No	Normal fare
TC2	10	200 km	AC3	No	50% concession
TC3	65	350 km	AC2	No	40% concession
TC4	30	250 km	Sleeper	Yes	Cancellation penalty applied
TC5	4	150 km	AC3	No	Free fare + reservation charge

9. Sample Output

```
RAILWAY FARE AND RESERVATION CALCULATOR


Enter number of passengers: 2


Passenger 1

Enter passenger name: Ravi

Enter age: 25

Enter travel distance (km): 250

Enter class (Sleeper/AC3/AC2): Sleeper

Is the ticket cancelled? (yes/no): no


--- FARE DETAILS ---

Passenger Name: Ravi

Age: 25

Distance: 250.0 km

Class: sleeper
```

Base Fare: ₹ 312.5

Reservation Charge: ₹ 50

Cancellation Penalty: ₹ 0

Final Fare: ₹ 362.5

Passenger 2

Enter passenger name: Sita

Enter age: 65

Enter travel distance (km): 350

Enter class (Sleeper/AC3/AC2): AC2

Is the ticket cancelled? (yes/no): no

--- FARE DETAILS ---

Passenger Name: Sita

Age: 65

Distance: 350.0 km

Class: ac2

Base Fare: ₹ 420.0

Reservation Charge: ₹ 50

Cancellation Penalty: ₹ 0

Final Fare: ₹ 470.0

Total Amount: ₹ 832.5

10. Conclusion

The Python program successfully calculates railway fares for multiple passengers by considering distance, travel class, age-based concessions, reservation charges, and cancellation penalties. It provides the final fare for each passenger and the total amount.

CASE STUDY 7: RETAIL BILLING WITH MULTIPLE DISCOUNT RULES

1. Problem Statement

Develop a Python program to process a retail bill containing multiple products and quantities. The program should calculate category-specific taxes, apply threshold-based discounts, enforce a maximum concession limit, and generate an itemized invoice.

2. Objectives

- To process multiple products and quantities.
- To calculate the cost of each item.
- To apply category-specific tax rates.
- To apply discounts based on the total purchase amount.
- To limit the maximum discount/concession.
- To generate an itemized invoice.
- To calculate the final payable amount.

3. Assumptions

The PDF does not specify numerical tax rates or discount values. The following assumptions are used to make the program runnable.

Product Categories

Category	Tax Rate
Grocery	5%
Electronics	18%
Clothing	12%

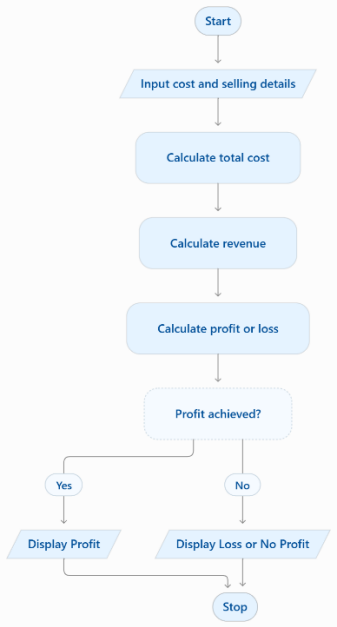
Discount Rules

- Bill above ₹5,000 → 10% discount.
- Bill from ₹2,500 to ₹5,000 → 5% discount.
- Bill below ₹2,500 → No discount.
- Maximum discount/concession = ₹1,000.
- Tax is calculated after discount.

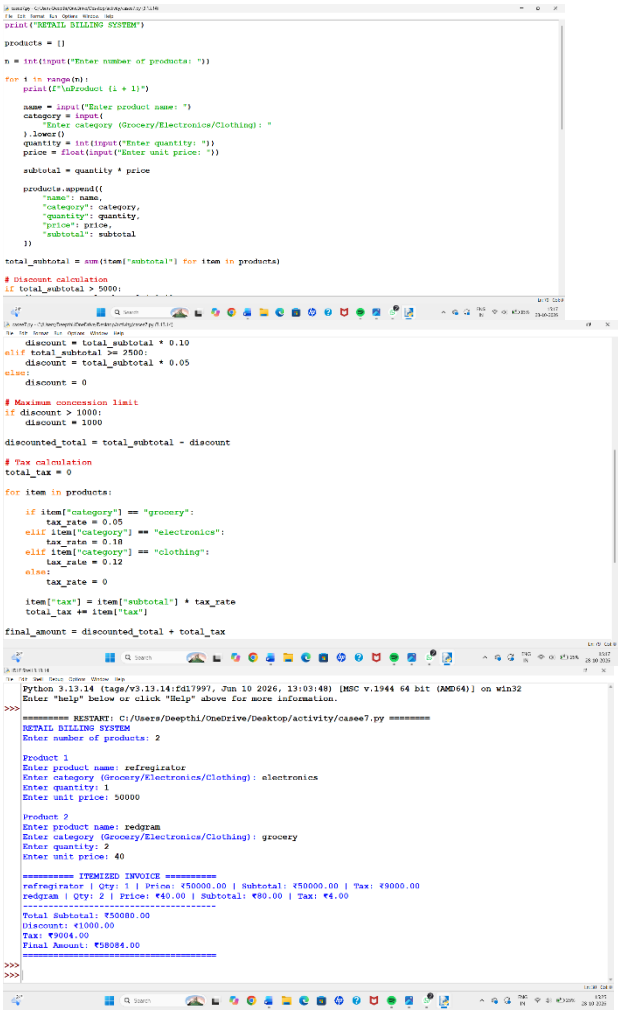
4. Algorithm

1. Start.
2. Read the number of products.
3. For each product, read:
    - Product name
    - Category
    - Quantity
    - Unit price
4. Calculate the subtotal for each product.
5. Calculate the total subtotal.
6. Apply the appropriate threshold discount.
7. Check the maximum discount limit.
8. Calculate category-specific tax.
9. Calculate the final bill amount.
10. Display the itemized invoice.
11. Stop.

5. Flowchart



7. Pseudocode



```
print("RETAIL BILLING SYSTEM")

products = []

n = int(input("Enter number of products: "))

for i in range(n):
    print(f"\nProduct {i + 1}")

    name = input("Enter product name: ")
    category = input(
        "Enter category (Grocery/Electronics/Clothing): "
    ).lower()
    quantity = int(input("Enter quantity: "))
    price = float(input("Enter unit price: "))

    subtotal = quantity * price

    products.append({
        "name": name,
        "category": category,
        "quantity": quantity,
        "price": price,
        "subtotal": subtotal
    })

total_subtotal = sum(item["subtotal"] for item in products)

# Discount calculation
if total_subtotal > 5000:
    discount = total_subtotal * 0.10
elif total_subtotal >= 2500:
    discount = total_subtotal * 0.05
else:
    discount = 0

# Maximum concession limit
if discount > 1000:
    discount = 1000

discounted_total = total_subtotal - discount

# Tax calculation
total_tax = 0

for item in products:

    if item["category"] == "grocery":
        tax_rate = 0.05
    elif item["category"] == "electronics":
        tax_rate = 0.18
    elif item["category"] == "clothing":
        tax_rate = 0.12
    else:
        tax_rate = 0

    item["tax"] = item["subtotal"] * tax_rate
    total_tax += item["tax"]

final_amount = discounted_total + total_tax
```

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ========== RESTART: C:/Users/Deepthi/OneDrive/Desktop/activity/casee7.py ========
RETAIL BILLING SYSTEM
Enter number of products: 2

Product 1
Enter product name: refregirator
Enter category (Grocery/Electronics/Clothing): electronics
Enter quantity: 1
Enter unit price: 50000

Product 2
Enter product name: redgram
Enter category (Grocery/Electronics/Clothing): grocery
Enter quantity: 2
Enter unit price: 40

========== ITEMIZED INVOICE ==========
refregirator | Qty: 1 | Price: ₹50000.00 | Subtotal: ₹50000.00 | Tax: ₹9000.00
redgram | Qty: 2 | Price: ₹40.00 | Subtotal: ₹80.00 | Tax: ₹4.00
---------------------------------------
Total Subtotal: ₹50080.00
Discount: ₹1000.00
Tax: ₹9004.00
Final Amount: ₹58084.00
=======================================
>>>
>>>
```

START

INPUT number_of_products

total_subtotal = 0

FOR each product:

INPUT product_name

INPUT category

INPUT quantity

INPUT unit_price

subtotal = quantity × unit_price

total_subtotal = total_subtotal + subtotal

IF total_subtotal > 5000:

discount = total_subtotal × 10%

ELSE IF total_subtotal >= 2500:

discount = total_subtotal × 5%

ELSE:

discount = 0

IF discount > 1000:

discount = 1000

discounted_amount = total_subtotal - discount

FOR each product:

Determine tax rate according to category

Calculate tax

total_tax = sum of all taxes

final_amount = discounted_amount + total_tax

DISPLAY itemized invoice

DISPLAY discount

DISPLAY tax

DISPLAY final amount

STOP

7. Python Source Code

```
print("RETAIL BILLING SYSTEM")

products = []

n = int(input("Enter number of products: "))

for i in range(n):
  print(f"\nProduct {i + 1}")

  name = input("Enter product name: ")
  category = input(
    "Enter category (Grocery/Electronics/Clothing): "
  ).lower()
  quantity = int(input("Enter quantity: "))
  price = float(input("Enter unit price: "))

  subtotal = quantity * price

  products.append({
```

```
        "name": name,
        "category": category,
        "quantity": quantity,
        "price": price,
        "subtotal": subtotal
    })

total_subtotal = sum(item["subtotal"] for item in products)

# Discount calculation
if total_subtotal > 5000:
    discount = total_subtotal * 0.10
elif total_subtotal >= 2500:
    discount = total_subtotal * 0.05
else:
    discount = 0

# Maximum concession limit
if discount > 1000:
    discount = 1000

discounted_total = total_subtotal - discount

# Tax calculation
total_tax = 0

for item in products:
```

```
    if item["category"] == "grocery":
        tax_rate = 0.05
    elif item["category"] == "electronics":
        tax_rate = 0.18
    elif item["category"] == "clothing":
        tax_rate = 0.12
    else:
        tax_rate = 0

    item["tax"] = item["subtotal"] * tax_rate
    total_tax += item["tax"]

final_amount = discounted_total + total_tax

print("\n========== ITEMIZED INVOICE ==========")

for item in products:
    print(
        f"{item['name']} | "
        f"Qty: {item['quantity']} | "
        f"Price: ₹{item['price']:.2f} | "
        f"Subtotal: ₹{item['subtotal']:.2f} | "
        f"Tax: ₹{item['tax']:.2f}"
    )

print("-------------------------------------")
print(f"Total Subtotal: ₹{total_subtotal:.2f}")
print(f"Discount: ₹{discount:.2f}")
```

print(f"Tax: ₹{total_tax:.2f}")

print(f"Final Amount: ₹{final_amount:.2f}")

print("==========")

8. Test Cases

Test Case	Products	Subtotal	Expected Discount	Expected Result
TC1	Grocery items	₹2,000	₹0	No discount
TC2	Clothing items	₹3,000	₹150	5% discount
TC3	Electronics + Grocery	₹6,000	₹600	10% discount
TC4	Electronics	₹15,000	₹1,000	Maximum discount applied

9. Sample Output

RETAIL BILLING SYSTEM

Enter number of products: 3

Product 1

Enter product name: Rice

Enter category (Grocery/Electronics/Clothing): Grocery

Enter quantity: 2

Enter unit price: 500

Product 2

Enter product name: Shirt

Enter category (Grocery/Electronics/Clothing): Clothing

Enter quantity: 2

Enter unit price: 800

Product 3

Enter product name: Headphones

Enter category (Grocery/Electronics/Clothing): Electronics

Enter quantity: 1

Enter unit price: 2000

========== ITEMIZED INVOICE ==========

Rice | Qty: 2 | Price: ₹500.00 | Subtotal: ₹1000.00 | Tax: ₹50.00

Shirt | Qty: 2 | Price: ₹800.00 | Subtotal: ₹1600.00 | Tax: ₹192.00

Headphones | Qty: 1 | Price: ₹2000.00 | Subtotal: ₹2000.00 | Tax: ₹360.00

Total Subtotal: ₹4600.00

Discount: ₹230.00

Tax: ₹602.00

Final Amount: ₹4972.00

==========

10. Conclusion

The Python program successfully processes multiple retail products, calculates individual subtotals, applies category-specific taxes and threshold-based discounts, limits the maximum concession, and generates an itemized invoice with the final payable amount.

CASE STUDY 8: MANUFACTURING COST AND PROFITABILITY ANALYSIS

1. Problem Statement

Develop a Python program to analyze the manufacturing cost of a product by considering material cost, labour cost, wastage, machine cost, and selling price. The program should calculate the total production cost, profit, profit percentage, and determine whether the production meets the specified profit target.

2. Objectives

- **To calculate material cost.**
- **To calculate labour cost.**
- **To calculate wastage cost.**
- **To include machine cost.**
- **To calculate total manufacturing cost.**
- **To calculate profit and profit percentage.**
- **To check whether the specified profit target is achieved.**

3. Assumptions

The PDF does not specify numerical rates for materials, labour, wastage, machine costs, or the profit target. The following assumptions are used to make the program runnable:

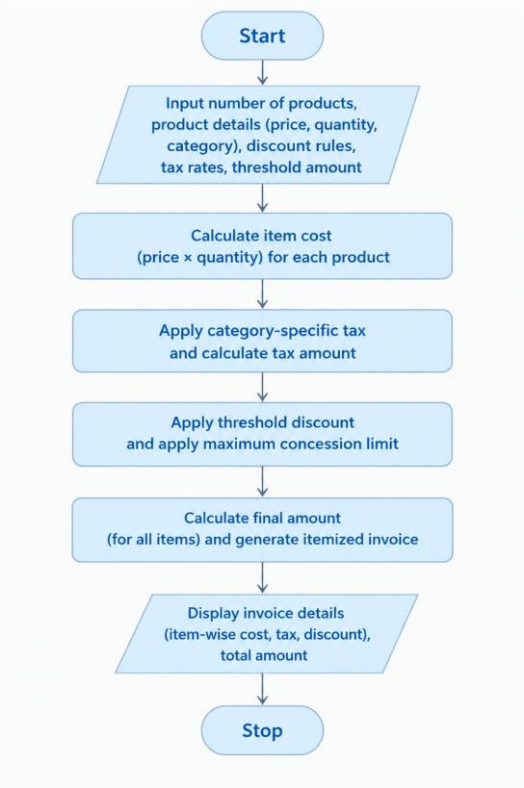
- **Material cost is entered per unit.**
- **Labour cost is entered per unit.**
- **Machine cost is entered per unit.**
- **Wastage percentage is applied to material cost.**
- **Selling price is entered per unit.**
- **Target profit percentage is entered by the user.**
- **Profit = Selling Price − Total Production Cost.**

4. Algorithm

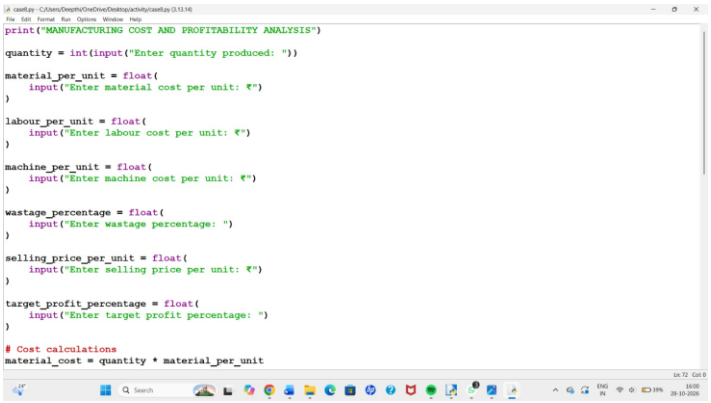
1. **Start.**
2. **Read the quantity produced.**
3. **Read material cost per unit.**
4. **Read labour cost per unit.**
5. **Read machine cost per unit.**
6. **Read wastage percentage.**
7. **Read selling price per unit.**
8. **Read target profit percentage.**
9. **Calculate total material cost.**

10. **Calculate wastage cost.**
11. **Calculate total labour cost.**
12. **Calculate total machine cost.**
13. **Calculate total production cost.**
14. **Calculate total sales value.**
15. **Calculate profit.**
16. **Calculate profit percentage.**
17. **Compare actual profit percentage with target profit percentage.**
18. **Display the result.**
19. **Stop.**

5. Flowchart



7. Pseudocode



```
print("MANUFACTURING COST AND PROFITABILITY ANALYSIS")

quantity = int(input("Enter quantity produced: "))

material_per_unit = float(
    input("Enter material cost per unit: ₹")
)

labour_per_unit = float(
    input("Enter labour cost per unit: ₹")
)

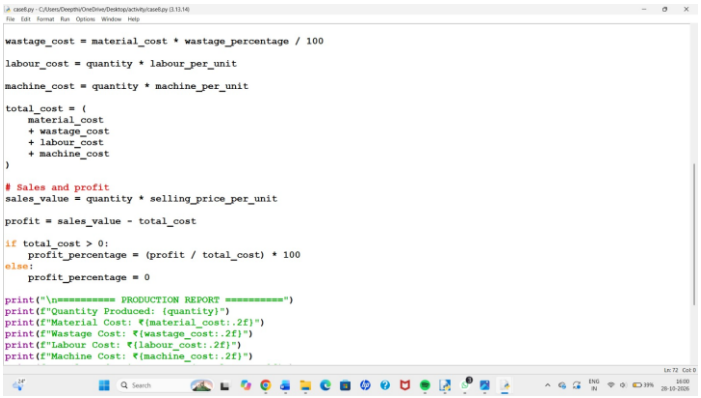
machine_per_unit = float(
    input("Enter machine cost per unit: ₹")
)

wastage_percentage = float(
    input("Enter wastage percentage: ")
)

selling_price_per_unit = float(
    input("Enter selling price per unit: ₹")
)

target_profit_percentage = float(
    input("Enter target profit percentage: ")
)

# Cost calculations
material_cost = quantity * material_per_unit
```



```
wastage_cost = material_cost * wastage_percentage / 100

labour_cost = quantity * labour_per_unit

machine_cost = quantity * machine_per_unit

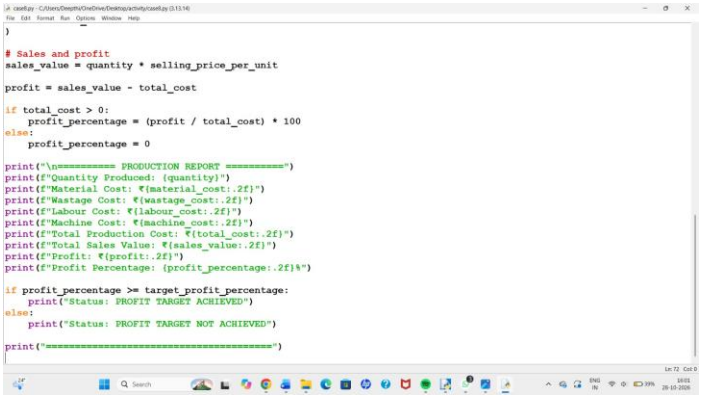
total_cost = (
    material_cost
    + wastage_cost
    + labour_cost
    + machine_cost
)

# Sales and profit
sales_value = quantity * selling_price_per_unit

profit = sales_value - total_cost

if total_cost > 0:
    profit_percentage = (profit / total_cost) * 100
else:
    profit_percentage = 0

print("\n========== PRODUCTION REPORT ==========")
print(f"Quantity Produced: {quantity}")
print(f"Material Cost: ₹{material_cost:.2f}")
print(f"Wastage Cost: ₹{wastage_cost:.2f}")
print(f"Labour Cost: ₹{labour_cost:.2f}")
print(f"Machine Cost: ₹{machine_cost:.2f}")
```



```
)

# Sales and profit
sales_value = quantity * selling_price_per_unit

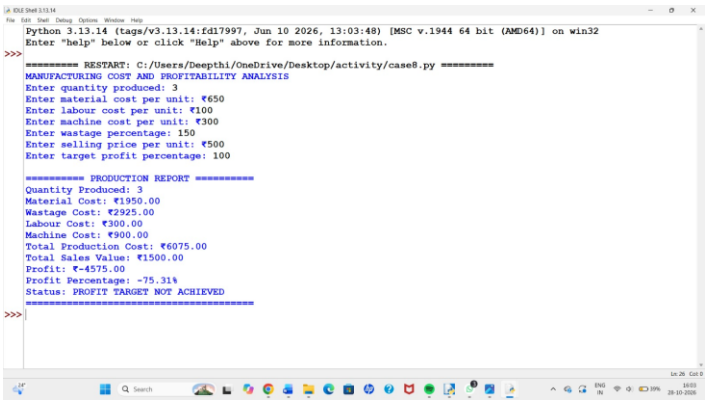
profit = sales_value - total_cost

if total_cost > 0:
    profit_percentage = (profit / total_cost) * 100
else:
    profit_percentage = 0

print("\n========== PRODUCTION REPORT ==========")
print(f"Quantity Produced: {quantity}")
print(f"Material Cost: ₹{material_cost:.2f}")
print(f"Wastage Cost: ₹{wastage_cost:.2f}")
print(f"Labour Cost: ₹{labour_cost:.2f}")
print(f"Machine Cost: ₹{machine_cost:.2f}")
print(f"Total Production Cost: ₹{total_cost:.2f}")
print(f"Total Sales Value: ₹{sales_value:.2f}")
print(f"Profit: ₹{profit:.2f}")
print(f"Profit Percentage: {profit_percentage:.2f}%")

if profit_percentage >= target_profit_percentage:
    print("Status: PROFIT TARGET ACHIEVED")
else:
    print("Status: PROFIT TARGET NOT ACHIEVED")

print("======================================")
```



```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/Deepthi/OneDrive/Desktop/activity/case8.py ==========
MANUFACTURING COST AND PROFITABILITY ANALYSIS
Enter quantity produced: 3
Enter material cost per unit: ₹650
Enter labour cost per unit: ₹100
Enter machine cost per unit: ₹300
Enter wastage percentage: 150
Enter selling price per unit: ₹500
Enter target profit percentage: 100

========== PRODUCTION REPORT ==========
Quantity Produced: 3
Material Cost: ₹1950.00
Wastage Cost: ₹2925.00
Labour Cost: ₹300.00
Machine Cost: ₹900.00
Total Production Cost: ₹6075.00
Total Sales Value: ₹1500.00
Profit: ₹-4575.00
Profit Percentage: -75.31%
Status: PROFIT TARGET NOT ACHIEVED
======================================
>>>
```

START

INPUT quantity

INPUT material_cost_per_unit

INPUT labour_cost_per_unit

INPUT machine_cost_per_unit

INPUT wastage_percentage

INPUT selling_price_per_unit

INPUT target_profit_percentage

material_cost = quantity × material_cost_per_unit

wastage_cost = material_cost × wastage_percentage / 100

labour_cost = quantity × labour_cost_per_unit

machine_cost = quantity × machine_cost_per_unit

total_cost = material_cost + wastage_cost
+ labour_cost + machine_cost

sales_value = quantity × selling_price_per_unit

profit = sales_value - total_cost

profit_percentage = (profit / total_cost) × 100

IF profit_percentage >= target_profit_percentage:

  Display "Profit Target Achieved"

ELSE:

  Display "Profit Target Not Achieved"

DISPLAY all cost and profit details

STOP

7. Python Source Code

```
print("MANUFACTURING COST AND PROFITABILITY ANALYSIS")

quantity = int(input("Enter quantity produced: "))

material_per_unit = float(
  input("Enter material cost per unit: ₹")
)

labour_per_unit = float(
  input("Enter labour cost per unit: ₹")
)

machine_per_unit = float(
  input("Enter machine cost per unit: ₹")
)

wastage_percentage = float(
  input("Enter wastage percentage: ")
)
```

```
selling_price_per_unit = float(
    input("Enter selling price per unit: ₹")
)

target_profit_percentage = float(
    input("Enter target profit percentage: ")
)

# Cost calculations
material_cost = quantity * material_per_unit

wastage_cost = material_cost * wastage_percentage / 100

labour_cost = quantity * labour_per_unit

machine_cost = quantity * machine_per_unit

total_cost = (
    material_cost
    + wastage_cost
    + labour_cost
    + machine_cost
)

# Sales and profit
sales_value = quantity * selling_price_per_unit
```

```
profit = sales_value - total_cost


if total_cost > 0:
   profit_percentage = (profit / total_cost) * 100
else:
   profit_percentage = 0


print("\n========== PRODUCTION REPORT ==========")
print(f"Quantity Produced: {quantity}")
print(f"Material Cost: ₹{material_cost:.2f}")
print(f"Wastage Cost: ₹{wastage_cost:.2f}")
print(f"Labour Cost: ₹{labour_cost:.2f}")
print(f"Machine Cost: ₹{machine_cost:.2f}")
print(f"Total Production Cost: ₹{total_cost:.2f}")
print(f"Total Sales Value: ₹{sales_value:.2f}")
print(f"Profit: ₹{profit:.2f}")
print(f"Profit Percentage: {profit_percentage:.2f}%")


if profit_percentage >= target_profit_percentage:
   print("Status: PROFIT TARGET ACHIEVED")
else:
   print("Status: PROFIT TARGET NOT ACHIEVED")


print("=======================================")
```

8. Test Cases

Test Case	Quantity	Material/Unit	Labour/Unit	Machine/Unit	Wastage	Selling Price	Target	Expected Result

TC1	**100**	**₹100**	**₹30**	**₹20**	**5%**	**₹200**	**20%**	**Target Achieved**
TC2	**50**	**₹150**	**₹40**	**₹30**	**5%**	**₹250**	**20%**	**Target Achieved**
TC3	**100**	**₹120**	**₹50**	**₹30**	**10%**	**₹200**	**25%**	**Target Not Achieved**

9. Sample Output

MANUFACTURING COST AND PROFITABILITY ANALYSIS

Enter quantity produced: 100

Enter material cost per unit: ₹100

Enter labour cost per unit: ₹30

Enter machine cost per unit: ₹20

Enter wastage percentage: 5

Enter selling price per unit: ₹200

Enter target profit percentage: 20

========== PRODUCTION REPORT ==========

Quantity Produced: 100

Material Cost: ₹10000.00

Wastage Cost: ₹500.00

Labour Cost: ₹3000.00

Machine Cost: ₹2000.00

Total Production Cost: ₹15500.00

Total Sales Value: ₹20000.00

Profit: ₹4500.00

Profit Percentage: 29.03%

Status: PROFIT TARGET ACHIEVED

10. Conclusion

The Python program successfully analyzes manufacturing costs by considering material, labour, wastage, and machine costs. It calculates the total production cost, sales value, profit, and profit percentage, and determines whether the specified profit target has been achieved.

CASE STUDY 9: MULTI-CURRENCY TRAVEL BUDGET ANALYSIS

1. Problem Statement

Develop a Python program to analyze travel expenses in multiple currencies. The program should convert expenses into a common currency using supplied exchange rates, include currency conversion fees, compare the total spending with the travel budget, and identify the largest expense category.

2. Objectives

- To accept travel expenses in different currencies.
- To convert different currencies into a common currency.
- To apply currency conversion fees.
- To calculate total travel expenditure.
- To compare expenditure with the given travel budget.
- To identify the largest expense category.

3. Assumptions

The PDF does not specify particular currencies, exchange rates, or conversion-fee values. The following assumptions are used to make the program runnable:

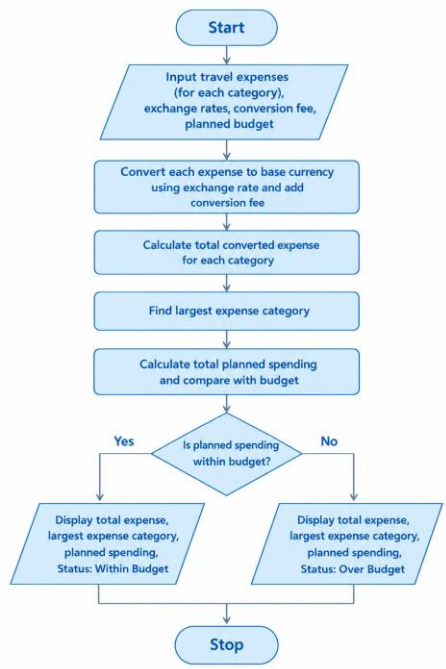
- The common currency is INR.
- Exchange rates are supplied by the user.
- Conversion fee is entered as a percentage.
- Expense categories include:
    - Accommodation
    - Food
    - Transport
    - Shopping
- The converted amount plus conversion fee is considered the final expense.
- The total expense is compared with the user's budget.

4. Algorithm

1. Start.

2. Read the travel budget in INR.
3. Read the conversion fee percentage.
4. Read the number of expenses.
5. For each expense:
    - Read expense category.
    - Read currency.
    - Read amount.
    - Read exchange rate.
6. Convert the amount into INR.
7. Calculate the conversion fee.
8. Add the fee to the converted amount.
9. Store the expense category and final amount.
10. Calculate the total travel expense.
11. Compare total expense with the budget.
12. Find the largest expense category.
13. Display the expense details and budget status.
14. Stop.

5. Flowchart



6.Pseudocode

```
print("MULTI-CURRENCY TRAVEL BUDGET ANALYSIS")

budget = float(input("Enter travel budget in INR: ₹"))

conversion_fee_percentage = float(
    input("Enter conversion fee percentage: ")
)

n = int(input("Enter number of expenses: "))

expenses = []

for i in range(n):
    print(f"\nExpense {i + 1}")

    category = input("Enter expense category: ")
    currency = input("Enter currency: ")
    amount = float(input("Enter amount: "))
    exchange_rate = float(
        input(f"Enter exchange rate of 1 {currency} in INR: ")
    )

    # Currency conversion
    converted_amount = amount * exchange_rate

    # Conversion fee
    conversion_fee = (
        converted_amount * conversion_fee_percentage / 100
    )
```

```
    final_amount = converted_amount + conversion_fee

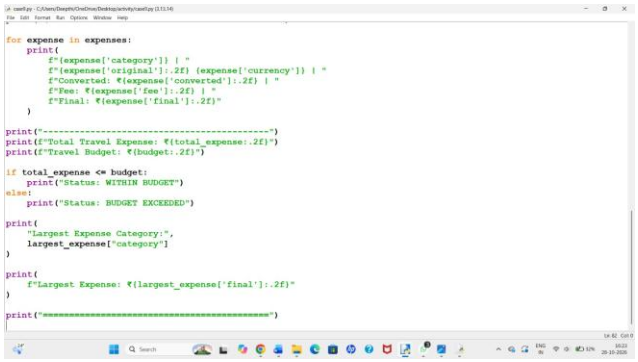
    expenses.append({
        "category": category,
        "currency": currency,
        "original": amount,
        "converted": converted_amount,
        "fee": conversion_fee,
        "final": final_amount
    })

total_expense = sum(
    expense["final"] for expense in expenses
)

# Find largest expense category
largest_expense = max(
    expenses,
    key=lambda x: x["final"]
)

print("\n========== TRAVEL EXPENSE REPORT ==========")

for expense in expenses:
    print(
        f"{expense['category']} | "
        f"{expense['original']:.2f} {expense['currency']} | "
        f"Converted: ₹{expense['converted']:.2f} | "
        f"Fee: ₹{expense['fee']:.2f} | "
        f"Final: ₹{expense['final']:.2f}"
```



```
for expense in expenses:
    print(
        f"{expense['category']} | "
        f"{expense['original']:.2f} {expense['currency']} | "
        f"Converted: ₹{expense['converted']:.2f} | "
        f"Fee: ₹{expense['fee']:.2f} | "
        f"Final: ₹{expense['final']:.2f}"
    )

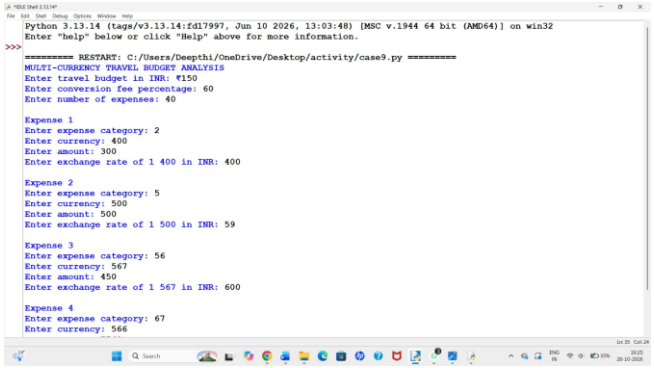
print("-----------------------------------------")
print(f"Total Travel Expense: ₹{total_expense:.2f}")
print(f"Travel Budget: ₹{budget:.2f}")

if total_expense <= budget:
    print("Status: WITHIN BUDGET")
else:
    print("Status: BUDGET EXCEEDED")

print(
    "Largest Expense Category:",
    largest_expense["category"]
)

print(
    f"Largest Expense: ₹{largest_expense['final']:.2f}"
)

print("=========================================")
```



```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========= RESTART: C:/Users/Deepthi/OneDrive/Desktop/activity/case9.py =========
MULTI-CURRENCY TRAVEL BUDGET ANALYSIS
Enter travel budget in INR: ₹150
Enter conversion fee percentage: 60
Enter number of expenses: 40

Expense 1
Enter expense category: 2
Enter currency: 400
Enter amount: 300
Enter exchange rate of 1 400 in INR: 400

Expense 2
Enter expense category: 5
Enter currency: 500
Enter amount: 500
Enter exchange rate of 1 500 in INR: 59

Expense 3
Enter expense category: 56
Enter currency: 567
Enter amount: 450
Enter exchange rate of 1 567 in INR: 600

Expense 4
Enter expense category: 67
Enter currency: 566
```

0START

INPUT travel_budget

INPUT conversion_fee_percentage

INPUT number_of_expenses

total_expense = 0

FOR each expense:

INPUT category

INPUT currency

INPUT amount

INPUT exchange_rate

converted_amount = amount × exchange_rate

conversion_fee =

converted_amount × conversion_fee_percentage / 100

final_amount = converted_amount + conversion_fee

Store category and final_amount

total_expense = total_expense + final_amount

Find the expense category with maximum amount

IF total_expense <= travel_budget:

Display "Within Budget"

ELSE:

Display "Budget Exceeded"

DISPLAY total expense

DISPLAY largest expense category

STOP

7. Python Source Code

```
print("MULTI-CURRENCY TRAVEL BUDGET ANALYSIS")

budget = float(input("Enter travel budget in INR: ₹"))

conversion_fee_percentage = float(
```

```
    input("Enter conversion fee percentage: ")
)

n = int(input("Enter number of expenses: "))

expenses = []

for i in range(n):
    print(f"\nExpense {i + 1}")

    category = input("Enter expense category: ")
    currency = input("Enter currency: ")
    amount = float(input("Enter amount: "))
    exchange_rate = float(
        input(f"Enter exchange rate of 1 {currency} in INR: ")
    )

    # Currency conversion
    converted_amount = amount * exchange_rate

    # Conversion fee
    conversion_fee = (
        converted_amount * conversion_fee_percentage / 100
    )

    final_amount = converted_amount + conversion_fee

    expenses.append({
```

```
        "category": category,
        "currency": currency,
        "original": amount,
        "converted": converted_amount,
        "fee": conversion_fee,
        "final": final_amount
    })

total_expense = sum(
    expense["final"] for expense in expenses
)

# Find largest expense category
largest_expense = max(
    expenses,
    key=lambda x: x["final"]
)

print("\n========== TRAVEL EXPENSE REPORT ==========")

for expense in expenses:
    print(
        f"{expense['category']} | "
        f"{expense['original']:.2f} {expense['currency']} | "
        f"Converted: ₹{expense['converted']:.2f} | "
        f"Fee: ₹{expense['fee']:.2f} | "
        f"Final: ₹{expense['final']:.2f}"
    )
```

```
print("-----------------------------------------")
print(f"Total Travel Expense: ₹{total_expense:.2f}")
print(f"Travel Budget: ₹{budget:.2f}")

if total_expense <= budget:
    print("Status: WITHIN BUDGET")
else:
    print("Status: BUDGET EXCEEDED")

print(
    "Largest Expense Category:",
    largest_expense["category"]
)

print(
    f"Largest Expense: ₹{largest_expense['final']:.2f}"
)

print("=========================================")
```

8. Test Cases

Test Case	Category	Currency	Amount	Exchange Rate	Expected Result
TC1	Accommodation	USD	100	₹85	Converted to INR
TC2	Food	EUR	50	₹92	Converted to INR
TC3	Transport	USD	40	₹85	Converted to INR
TC4	Shopping	GBP	30	₹110	Converted to INR

TC5	Multiple expenses	Mixed	—	—	Largest category identified

9. Sample Output

MULTI-CURRENCY TRAVEL BUDGET ANALYSIS

Enter travel budget in INR: ₹25000

Enter conversion fee percentage: 2

Enter number of expenses: 4

Expense 1

Enter expense category: Accommodation

Enter currency: USD

Enter amount: 100

Enter exchange rate of 1 USD in INR: 85

Expense 2

Enter expense category: Food

Enter currency: EUR

Enter amount: 50

Enter exchange rate of 1 EUR in INR: 92

Expense 3

Enter expense category: Transport

Enter currency: USD

Enter amount: 40

Enter exchange rate of 1 USD in INR: 85

Expense 4

Enter expense category: Shopping

Enter currency: GBP

Enter amount: 30

Enter exchange rate of 1 GBP in INR: 110

========== TRAVEL EXPENSE REPORT ==========

Accommodation | 100.00 USD | Converted: ₹8500.00 | Fee: ₹170.00 | Final: ₹8670.00

Food | 50.00 EUR | Converted: ₹4600.00 | Fee: ₹92.00 | Final: ₹4692.00

Transport | 40.00 USD | Converted: ₹3400.00 | Fee: ₹68.00 | Final: ₹3468.00

Shopping | 30.00 GBP | Converted: ₹3300.00 | Fee: ₹66.00 | Final: ₹3366.00

Total Travel Expense: ₹20196.00

Travel Budget: ₹25000.00

Status: WITHIN BUDGET

Largest Expense Category: Accommodation

Largest Expense: ₹8670.00

==========

10. Conclusion

The Python program successfully analyzes travel expenses in multiple currencies by converting them into INR, applying conversion fees, calculating total expenditure, comparing it with the travel budget, and identifying the largest expense category.

CASE STUDY 10: CAMPUS EVENT FINANCIAL PLANNING

1. Problem Statement

Develop a Python program to analyze the financial requirements of a campus event. The program should calculate venue, equipment, catering, transport, and certificate costs. It should also consider participant contributions and sponsorship, calculate any

sponsorship shortfall, and determine the minimum number of registrations required to break even.

2. Objectives

- To calculate the total venue cost.
- To calculate equipment expenses.
- To calculate catering expenses.
- To calculate transportation expenses.
- To calculate certificate costs.
- To calculate the total event expenditure.
- To calculate participant contributions.
- To determine sponsorship shortfall.
- To calculate the minimum number of registrations required to break even.

3. Assumptions

The PDF does not specify numerical costs or contribution amounts. The following values are therefore used as assumptions for a runnable program:

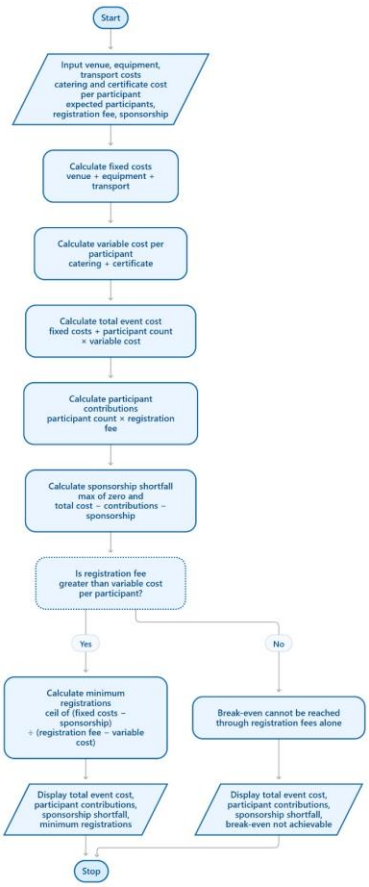
- Venue cost is entered by the user.
- Equipment cost is entered by the user.
- Transport cost is entered by the user.
- Certificate cost is entered per participant.
- Catering cost is entered per participant.
- Registration fee is the amount contributed by each participant.
- Sponsorship amount is entered by the user.
- Break-even registration count is calculated based on the total event cost after sponsorship.

4. Algorithm

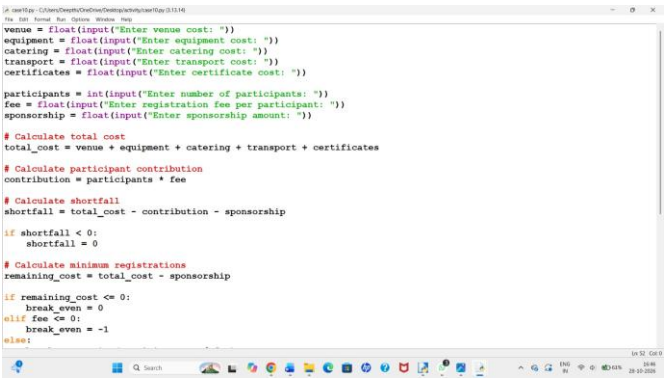
1. Start.
2. Read venue cost.
3. Read equipment cost.
4. Read transport cost.

5. Read catering cost per participant.
6. Read certificate cost per participant.
7. Read registration fee per participant.
8. Read sponsorship amount.
9. Read expected number of participants.
10. Calculate catering cost.
11. Calculate certificate cost.
12. Calculate total event cost.
13. Calculate the amount remaining after sponsorship.
14. Calculate participant contribution.
15. Calculate total available funds.
16. Determine whether there is a sponsorship shortfall.
17. Calculate the minimum number of registrations required to break even.
18. Display the complete financial report.
19. Stop.

5. Flowchart



6. Pseudocode



```
venue = float(input("Enter venue cost: "))
equipment = float(input("Enter equipment cost: "))
catering = float(input("Enter catering cost: "))
transport = float(input("Enter transport cost: "))
certificates = float(input("Enter certificate cost: "))

participants = int(input("Enter number of participants: "))
fee = float(input("Enter registration fee per participant: "))
sponsorship = float(input("Enter sponsorship amount: "))

# Calculate total cost
total_cost = venue + equipment + catering + transport + certificates

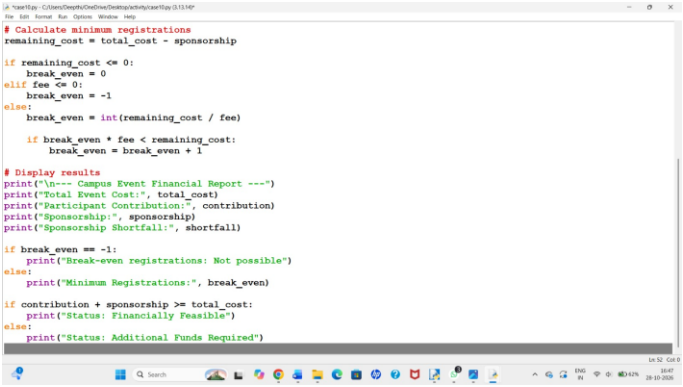
# Calculate participant contribution
contribution = participants * fee

# Calculate shortfall
shortfall = total_cost - contribution - sponsorship

if shortfall < 0:
    shortfall = 0

# Calculate minimum registrations
remaining_cost = total_cost - sponsorship

if remaining_cost <= 0:
    break_even = 0
elif fee <= 0:
    break_even = -1
else:
```



```
# Calculate minimum registrations
remaining_cost = total_cost - sponsorship

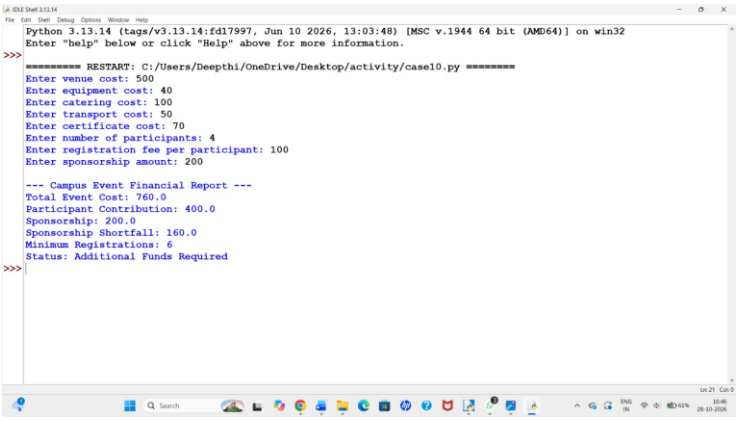
if remaining_cost <= 0:
    break_even = 0
elif fee <= 0:
    break_even = -1
else:
    break_even = int(remaining_cost / fee)

    if break_even * fee < remaining_cost:
        break_even = break_even + 1

# Display results
print("\n--- Campus Event Financial Report ---")
print("Total Event Cost:", total_cost)
print("Participant Contribution:", contribution)
print("Sponsorship:", sponsorship)
print("Sponsorship Shortfall:", shortfall)

if break_even == -1:
    print("Break-even registrations: Not possible")
else:
    print("Minimum Registrations:", break_even)

if contribution + sponsorship >= total_cost:
    print("Status: Financially Feasible")
else:
    print("Status: Additional Funds Required")
```



START

INPUT venue_cost

INPUT equipment_cost

INPUT transport_cost

INPUT catering_per_person

INPUT certificate_per_person

INPUT registration_fee

INPUT sponsorship

INPUT expected_participants

catering_cost =

  catering_per_person × expected_participants

certificate_cost =

  certificate_per_person × expected_participants

total_cost =

  venue_cost

  + equipment_cost

```
    + transport_cost
    + catering_cost
    + certificate_cost

participant_contribution =
    registration_fee × expected_participants

available_funds =
    sponsorship + participant_contribution

IF sponsorship < total_cost:
    sponsorship_shortfall =
        total_cost - sponsorship
ELSE:
    sponsorship_shortfall = 0

amount_to_cover =
    total_cost - sponsorship

IF amount_to_cover <= 0:
    break_even_registrations = 0
ELSE:
    break_even_registrations =
        CEILING(amount_to_cover / registration_fee)

IF expected_participants >= break_even_registrations:
    Display "Event can break even"
ELSE:
```

Display "More registrations required"

DISPLAY financial report

STOP

7. Python Source Code

```
import math

print("CAMPUS EVENT FINANCIAL PLANNING")

venue_cost = float(input("Enter venue cost: ₹"))
equipment_cost = float(input("Enter equipment cost: ₹"))
transport_cost = float(input("Enter transport cost: ₹"))

catering_per_person = float(
    input("Enter catering cost per participant: ₹")
)

certificate_per_person = float(
    input("Enter certificate cost per participant: ₹")
)

registration_fee = float(
    input("Enter registration fee per participant: ₹")
)

sponsorship = float(
    input("Enter sponsorship amount: ₹")
```

```
)

expected_participants = int(
    input("Enter expected number of participants: ")
)

# Variable costs
catering_cost = (
    catering_per_person * expected_participants
)

certificate_cost = (
    certificate_per_person * expected_participants
)

# Total event cost
total_cost = (
    venue_cost
    + equipment_cost
    + transport_cost
    + catering_cost
    + certificate_cost
)

# Participant contribution
participant_contribution = (
    registration_fee * expected_participants
)
```

```
# Total available funds
available_funds = (
    sponsorship + participant_contribution
)

# Sponsorship shortfall
if sponsorship < total_cost:
    sponsorship_shortfall = total_cost - sponsorship
else:
    sponsorship_shortfall = 0

# Break-even registrations
amount_to_cover = total_cost - sponsorship

if amount_to_cover <= 0:
    break_even_registrations = 0
elif registration_fee > 0:
    break_even_registrations = math.ceil(
        amount_to_cover / registration_fee
    )
else:
    break_even_registrations = 0

print("\n========== EVENT FINANCIAL REPORT ==========")

print(f"Venue Cost: ₹{venue_cost:.2f}")
print(f"Equipment Cost: ₹{equipment_cost:.2f}")
```

```
print(f"Transport Cost: ₹{transport_cost:.2f}")
print(f"Catering Cost: ₹{catering_cost:.2f}")
print(f"Certificate Cost: ₹{certificate_cost:.2f}")

print("-----------------------------------------")
print(f"Total Event Cost: ₹{total_cost:.2f}")
print(f"Sponsorship: ₹{sponsorship:.2f}")
print(
   f"Participant Contribution: "
   f"₹{participant_contribution:.2f}"
)
print(f"Available Funds: ₹{available_funds:.2f}")
print(
   f"Sponsorship Shortfall: "
   f"₹{sponsorship_shortfall:.2f}"
)

print(
   f"Minimum Registrations for Break-Even: "
   f"{break_even_registrations}"
)

if expected_participants >= break_even_registrations:
   print("Status: EVENT CAN BREAK EVEN")
else:
   additional = (
      break_even_registrations - expected_participants
   )
```

```
    print("Status: MORE REGISTRATIONS REQUIRED")
    print(f"Additional Registrations Needed: {additional}")

print("==========================================")
```

8. Test Cases

Test Case	Expected Participants	Registration Fee	Sponsorship	Expected Result
TC1	100	₹500	₹20,000	Break-even calculation
TC2	50	₹400	₹10,000	More registrations may be required
TC3	200	₹600	₹30,000	Event can break even
TC4	20	₹300	₹5,000	Check sponsorship shortfall

9. Sample Output

```
CAMPUS EVENT FINANCIAL PLANNING

Enter venue cost: ₹20000
Enter equipment cost: ₹10000
Enter transport cost: ₹5000
Enter catering cost per participant: ₹200
Enter certificate cost per participant: ₹50
Enter registration fee per participant: ₹500
Enter sponsorship amount: ₹20000
Enter expected number of participants: 100

========== EVENT FINANCIAL REPORT ==========
Venue Cost: ₹20000.00
Equipment Cost: ₹10000.00
```

Transport Cost: ₹5000.00

Catering Cost: ₹20000.00

Certificate Cost: ₹5000.00

Total Event Cost: ₹60000.00

Sponsorship: ₹20000.00

Participant Contribution: ₹50000.00

Available Funds: ₹70000.00

Sponsorship Shortfall: ₹40000.00

Minimum Registrations for Break-Even: 80

Status: EVENT CAN BREAK EVEN

==========

10. Conclusion

The Python program successfully analyzes the financial requirements of a campus event by calculating venue, equipment, catering, transport, and certificate costs. It also considers sponsorship and participant contributions, identifies the sponsorship shortfall, and calculates the minimum number of registrations required to achieve break-even.